SPACE COMPLEXITY
INTRO TO BINARY SEARCH TREES

Problem Solving with Computers-II

Space Complexity

S(n) = auxiliary memory needed to compute F(n)

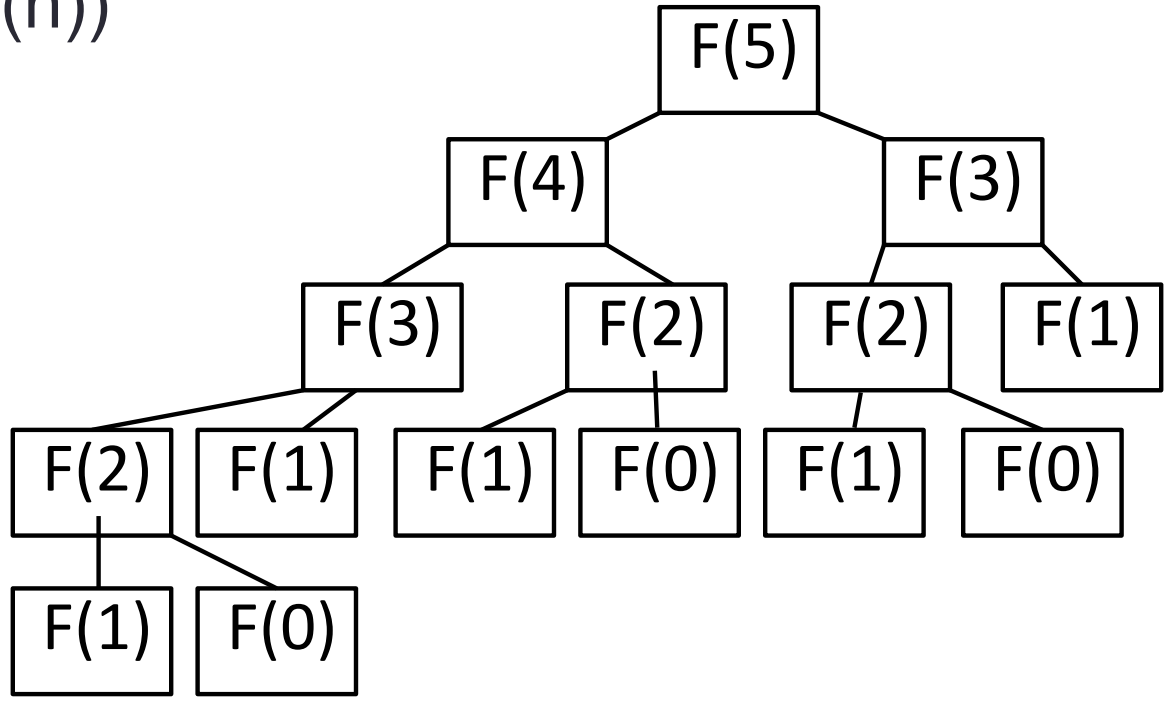
Auxiliary space: Peak space usage not considering input and output

```
F(int n){
  if(n <= 1) return 1
  return F(n-1) + F(n-2)
}
```

What is S(n)? Express your answer in Big-O notation

What is S(n)? Express your answer in Big-O notation

A. O(1)
B. O(log(n))
C. O(n)
D. $O(n^2)$
E. $O(2^n)$



Tree of recursive calls needed to compute F(5)

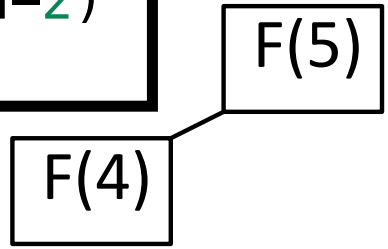
S(n) relates to maximum depth of the recursion

```
F(int n){
   if(n <= 1) return 1
   return F(n-1) + F(n-2)
}
```

F(5)

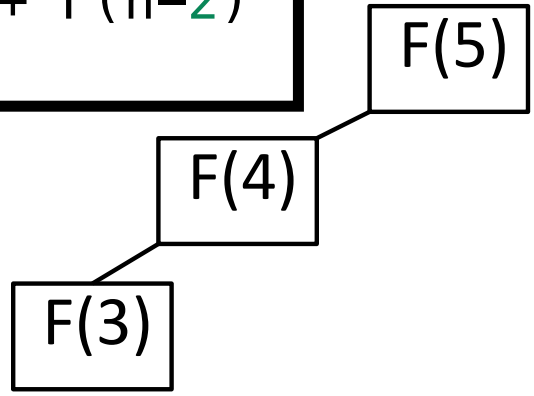
S(n) relates to maximum depth of the recursion

```
F(int n){
   if(n <= 1) return 1
   return F(n-1) + F(n-2)
}
```



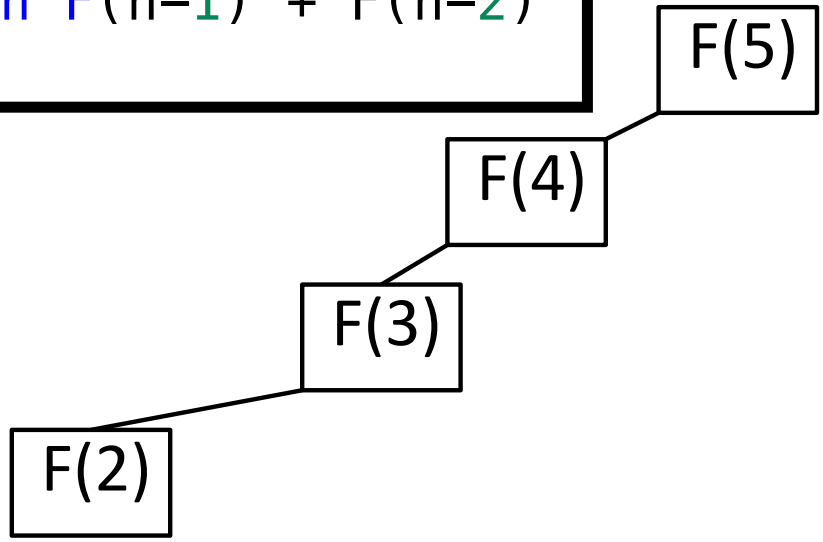
S(n) relates to maximum depth of the recursion

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



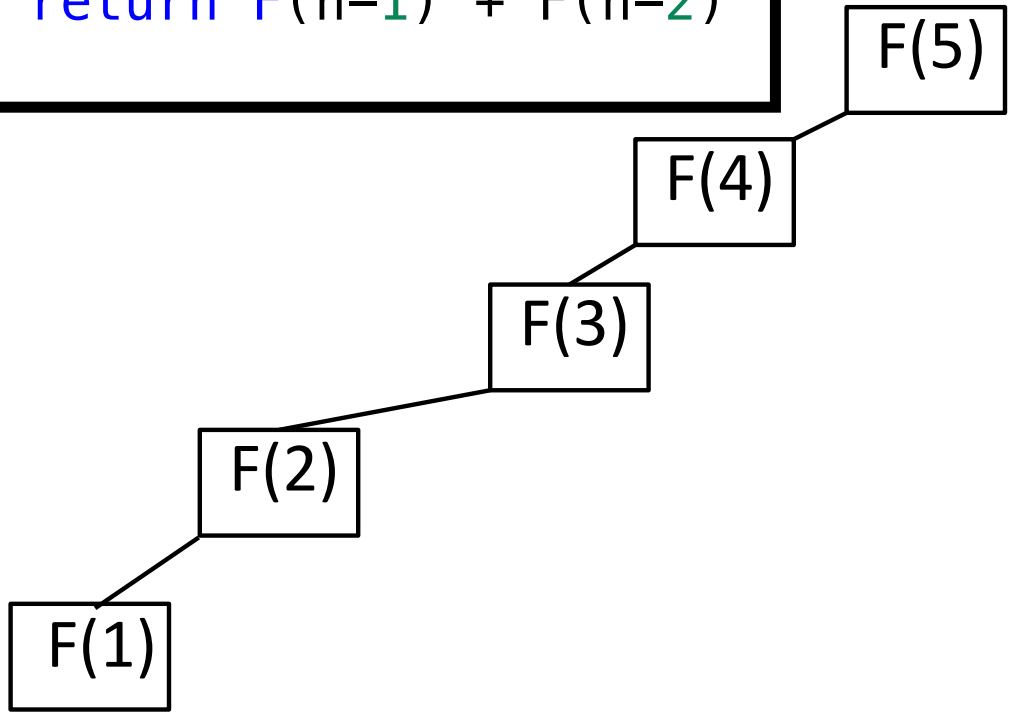
S(n) relates to maximum depth of the recursion

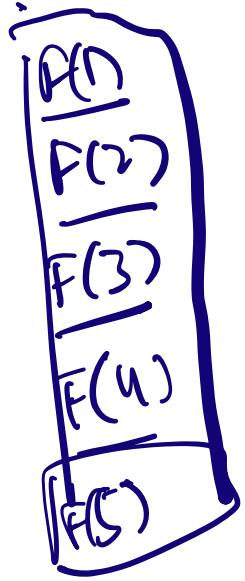
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



S(n) relates to maximum depth of the recursion

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```

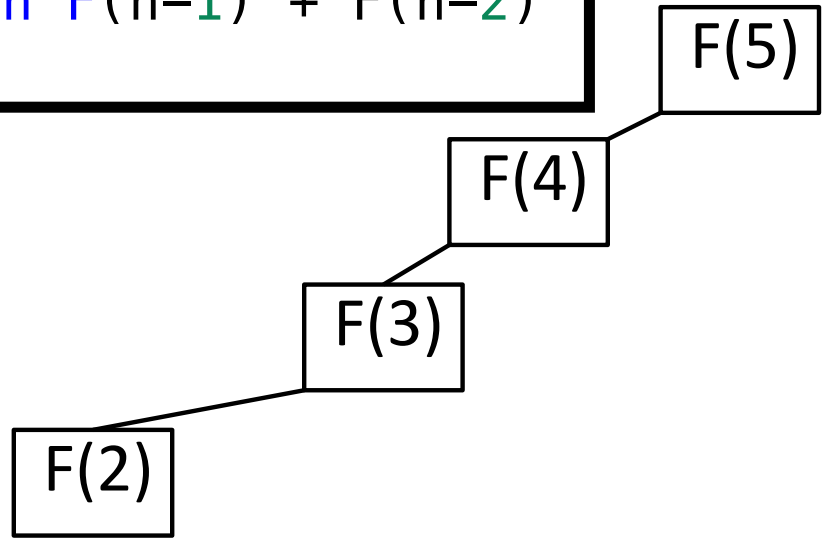




Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

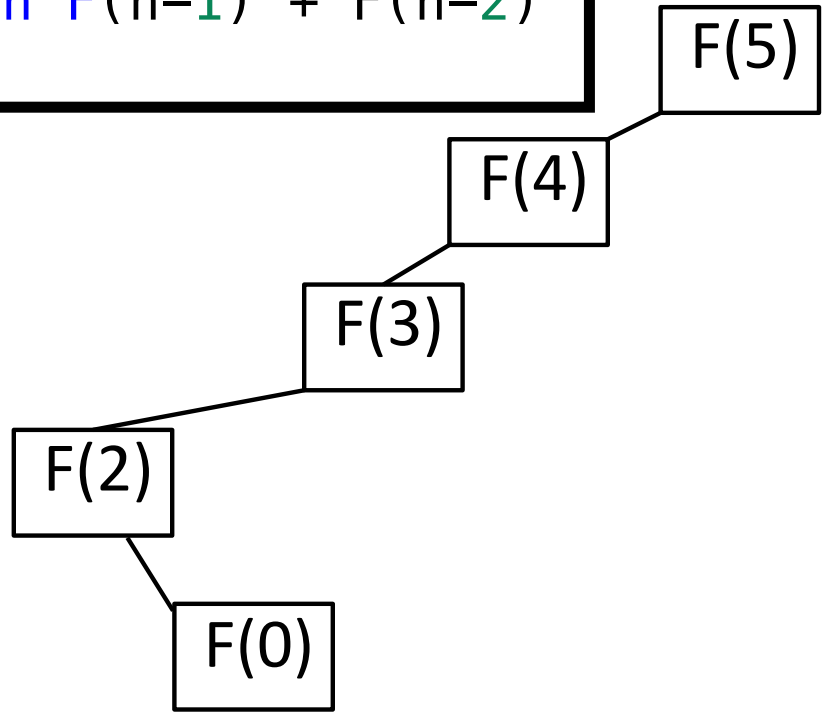
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

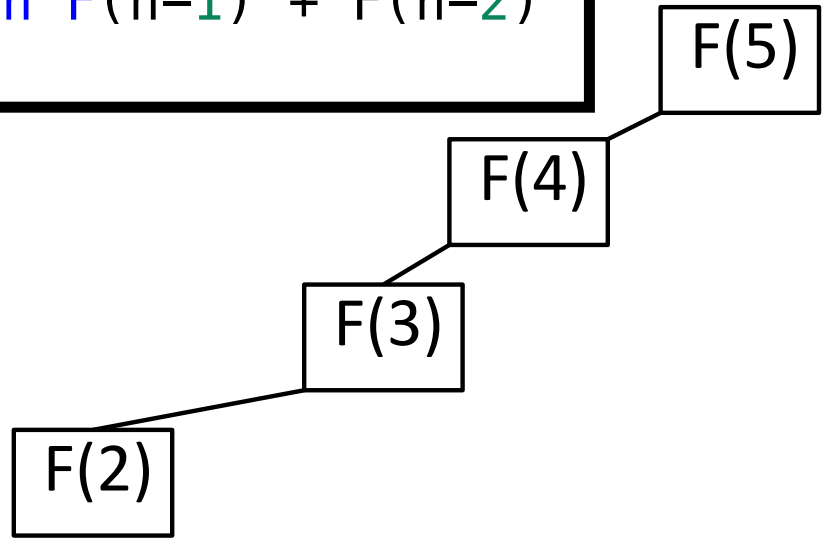
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

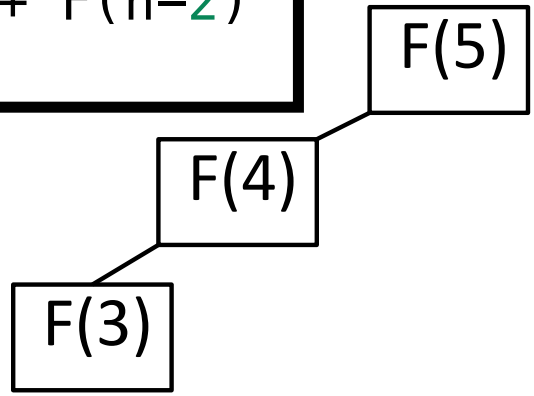
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

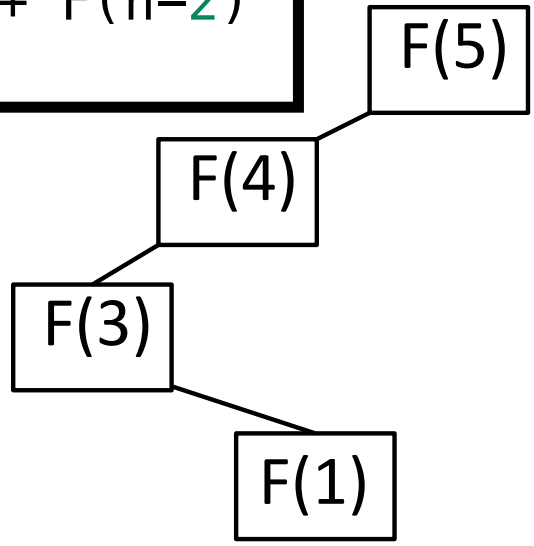
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

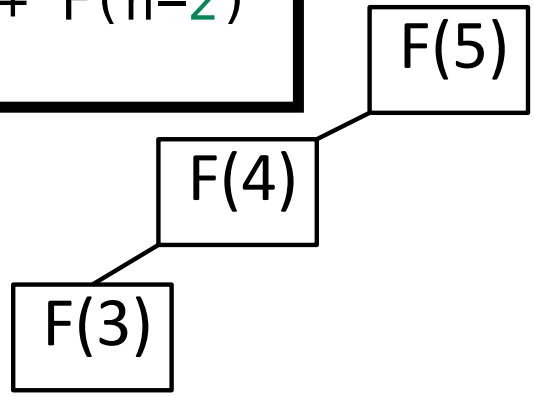
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

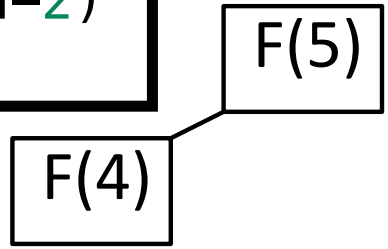
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

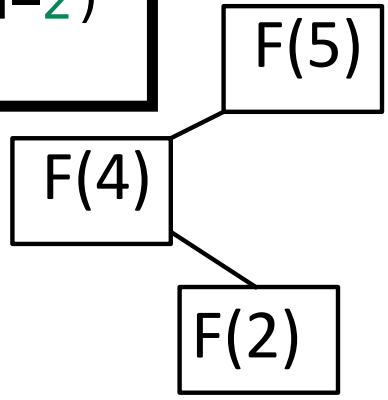
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

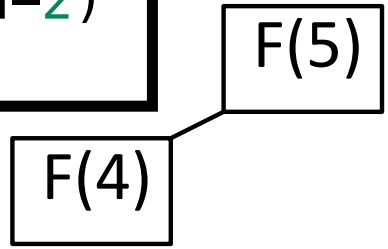
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

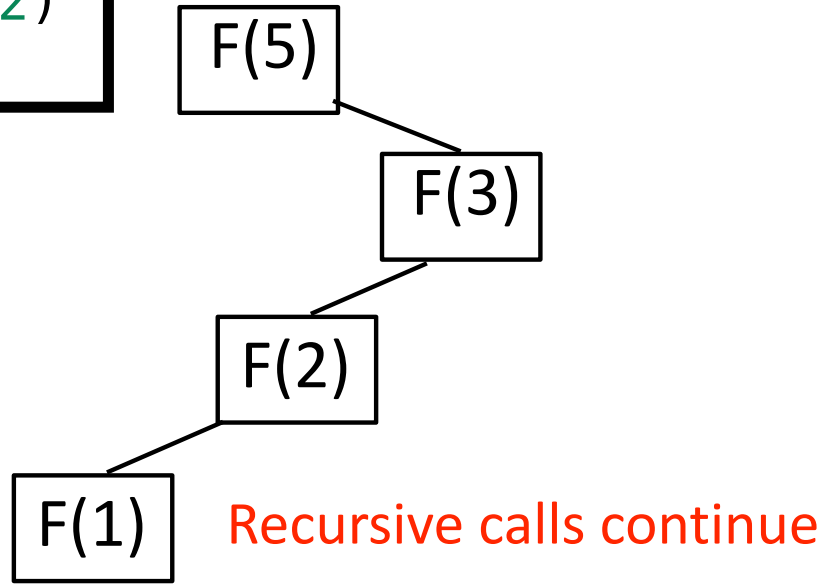
```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion = 5

S(n) relates to maximum depth of the recursion

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```



Maximum depth of the recursion for F(n) = n

Therefore, S(n) = O(n)

Which algorithm is more space efficient?

A.

```
F(int n){
    if(n <= 1) return 1
    return F(n-1) + F(n-2)
}
```

$S(n) = O(n)$ = max depth of recursion

B.

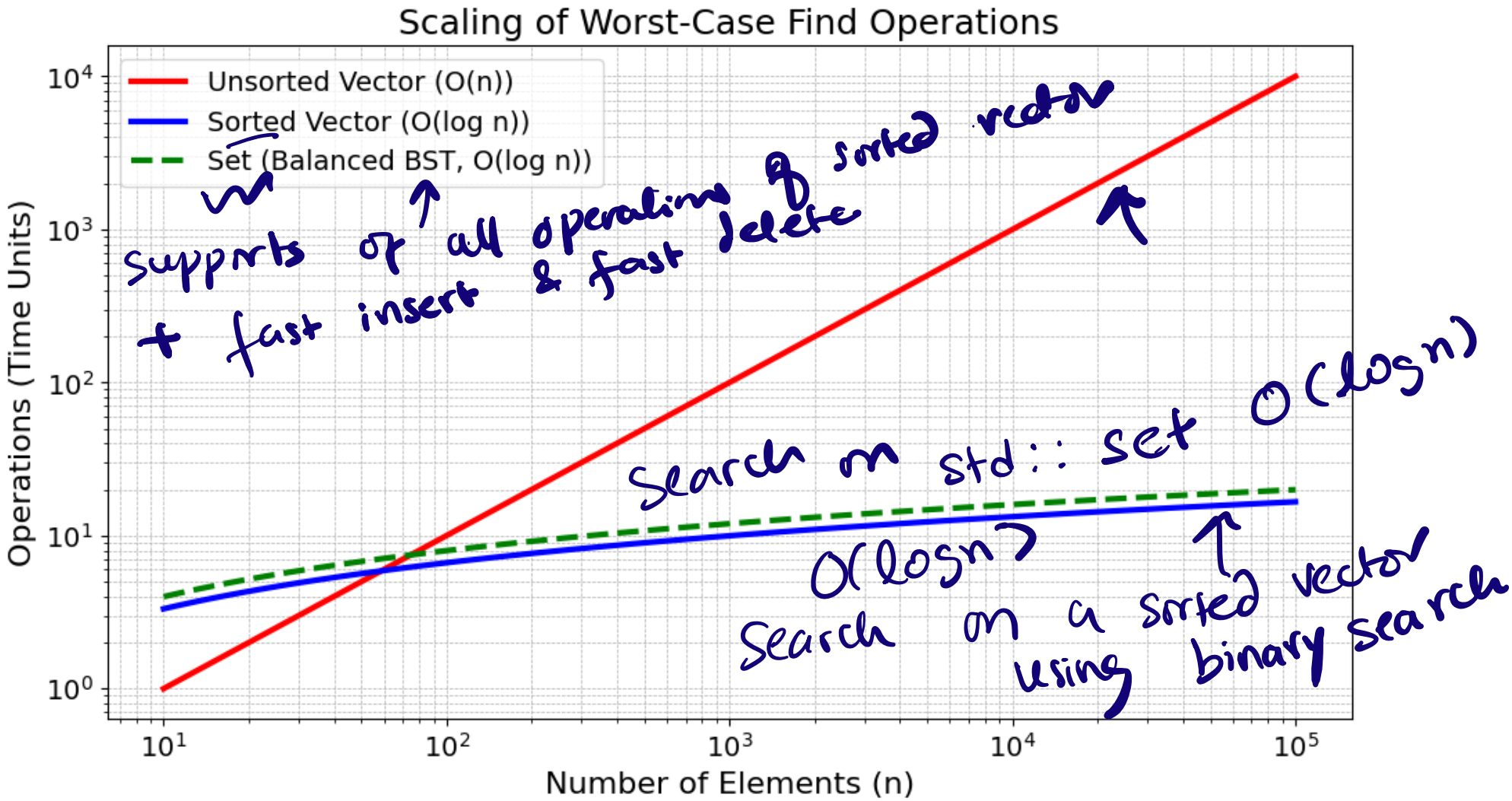
```
F(int n){
    Initialize A[0 . . . n]
    A[0] = A[1] = 1

    for i = 2 : n
      A[i] = A[i-1] + A[i-2]

    return A[n]
}
```

$S(n) = O(n)$

C. *Both are the same: O(n)*

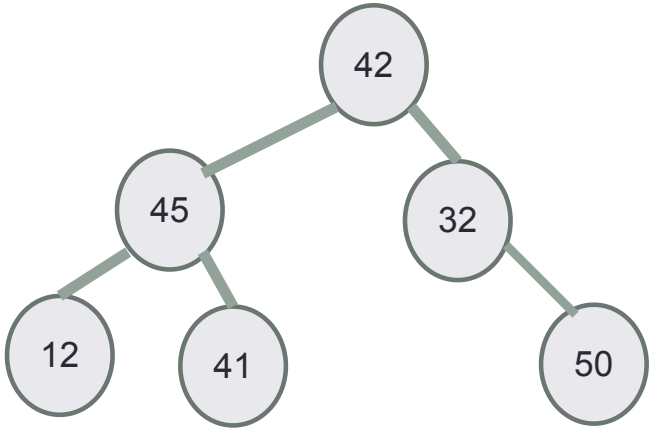

Scaling of Worst-Case Find Operations
Unsorted Vector (O(n))
Sorted Vector (O(log n))
Set (Balanced BST, O(log n))
Operations (Time Units)
Number of Elements (n)
10^0
10^1
10^2
10^3
10^4
10^5
supports of all operations of sorted vector
+ fast insert & fast delete
Search on std::set O(logn)
O(logn)
Search on a sorted vector using binary search

Binary Search Trees (std::set)

- What are the operations supported?
- What are the running times of these operations?
- How do you implement the BST i.e. operations supported by it?

https://cplusplus.com/reference/set/set/?kw=set

Binary Trees



In a tree, nodes are arranged in a hierarchy

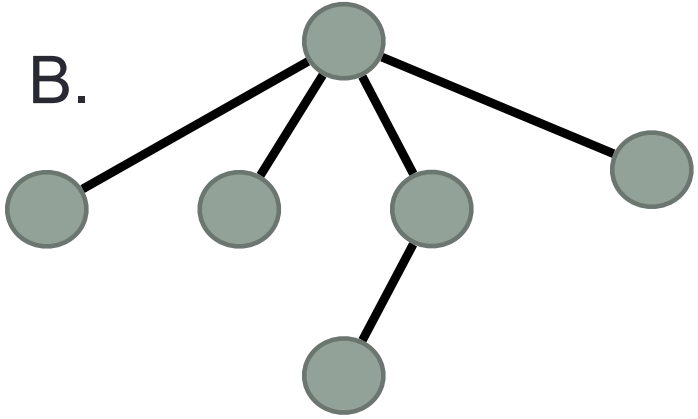
- One node is distinguished as the root
- Each node:
  - stores a key
  - has a pointer to child nodes and parent (optional)
- Unique path between any two nodes
- Leaf nodes have no children

In a binary tree, each node has at most two children

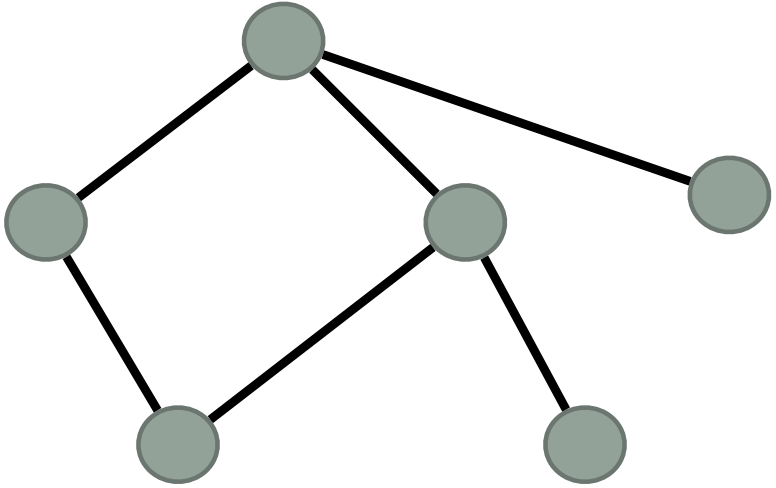
Which of the following is/are a tree?

A.

B.



C.



D. A & B

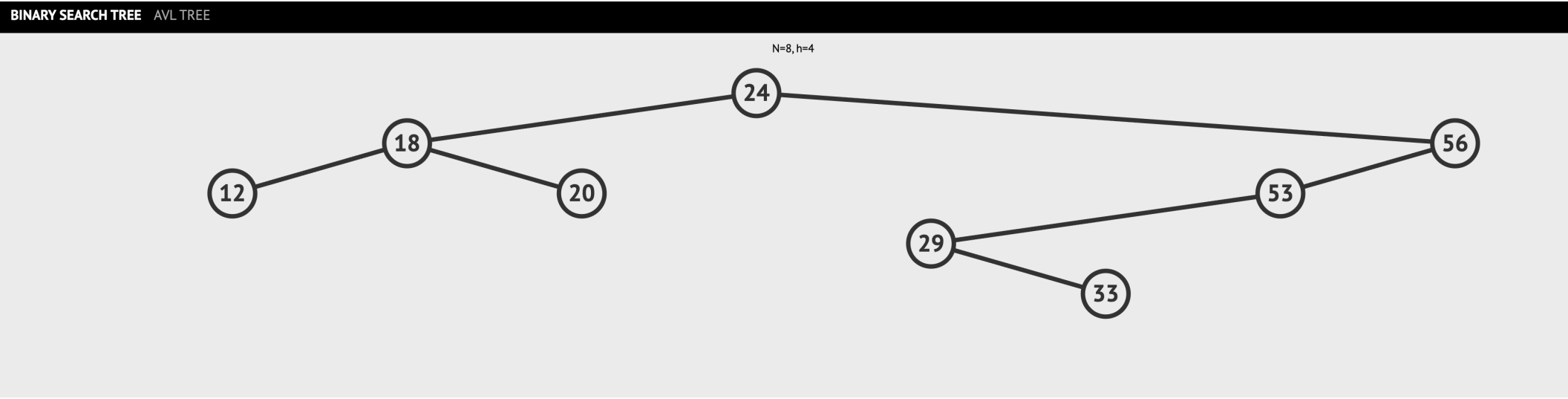
E. All of A-C

```
struct TreeNode {

  TreeNode* left;
  TreeNode* right;
  TreeNode* parent;
  int const data;

  TreeNode(int d) : data(d) {
    left = right = parent = nullptr;
  }
};
```

Binary Search Tree Visualization



Visualization: https://visualgo.net/en/bst